Resumen para el parcial práctico

> Para tener abierto **durante** el parcial: solo patrones, moldes de código y checklists. Las explicaciones están en el libro, Parcial-1-LeerAntesDePracticar.md. Cada "Receta N" de acá tiene su ejercicio resuelto en la receta con el mismo número del capítulo 8 del libro.
>
> Base: datos de 2010 a 2012. Si piden "2025" o "actualmente", probá con 2012 y en el parcial escribí el año que piden (o `YEAR(GETDATE())`).

1 — Joins y dónde vive cada dato

```
-- renglón → ticket (clave de 3 partes, SIEMPRE las tres)
JOIN Factura f ON f.fact_tipo = i.item_tipo AND f.fact_sucursal = i.item_sucursal AND f.fact_numero = i.item_numero
-- renglón → producto
JOIN Producto p ON p.prod_codigo = i.item_producto
-- producto → stock
JOIN STOCK s ON s.stoc_producto = p.prod_codigo
```

Necesito...	**Sale de...**
Unidades vendidas	`SUM(item_cantidad)`
Monto por factura / cliente	`SUM(fact_total)`
Monto por producto	`SUM(item_cantidad * item_precio)` (**nunca** `fact_total` agrupando por producto)
Fecha, cliente, vendedor	`Factura` (desde un renglón, hay que unir con `Factura`)
¿Es compuesto?	`prod_codigo IN (SELECT comp_producto FROM Composicion)`
Cantidad de depósitos	`(SELECT COUNT(*) FROM DEPOSITO)`
"Tiene stock"	`stoc_cantidad > 0` (no alcanza con tener fila en `STOCK`)

2 — Los 5 pasos

1. **Sujeto:** ¿una fila por qué cosa? → el FROM arranca en esa tabla.
2. **Columnas:** ¿sale del mismo FROM y filtro? → agregado (mesa). ¿Otro filtro, otro año, otra tabla hija, "el que más"? → subconsulta en el SELECT (mandadero).
3. **Filtros:** por fila → `WHERE`; por total → `HAVING`; "tiene / no tiene / alguna vez" → `[NOT] EXISTS`; "está entre los N que..." → `IN (SELECT TOP N ...)`.
4. **Orden:** `ORDER BY` admite `CASE` y subconsultas.
5. **Cardinalidad:** ¿uní el sujeto con **dos tablas hijas**? → `SUM` / `AVG` / `COUNT` salen inflados (`MIN`, `MAX` y `COUNT(DISTINCT)` no). Una hija en el FROM, la otra por mandadero.

```
SELECT
   s.codigo,
   s.detalle,
   AGREGADO(...)                          AS col_de_la_mesa,
   ( SELECT ... WHERE x.fk = s.codigo AND ... )    AS col_mandadero
FROM TablaDelSujeto s
[JOIN tabla_hija_principal ...]
WHERE  condicion_por_fila
  AND  [NOT] EXISTS ( ... correlacionada con s ... )
  AND  s.codigo IN ( SELECT TOP N ... )
GROUP BY s.codigo, s.detalle
HAVING  condicion_sobre_el_grupo
ORDER BY ...
```

3 — Self-joins

Si dice...	Cómo se arma
"jefe", "a cargo de"	`Empleado e JOIN Empleado j ON j.empl_codigo = e.empl_jefe`
combo con datos de los dos productos	`Composicion c JOIN Producto combo ON combo.prod_codigo = c.comp_producto JOIN Producto comp ON comp.prod_codigo = c.comp_componente`
"pares", "vendidos juntos", "en la misma factura"	`Item_Factura ia JOIN Item_Factura ib` por las 3 columnas `AND ib.item_producto > ia.item_producto`
"años consecutivos", "año anterior"	**no** self-join: año ancla (receta 4)

- Pares **sin repetir** → `>` (con `<>` sale (A,B) y (B,A); sin nada sale (A,A)).
- Alias con el nombre del rol (`e` / `j` , `ia` / `ib`), nunca `t1` / `t2` .
- `COUNT(*)` en un self-join de renglones cuenta pares; un `SUM` de un lado sale multiplicado → mandadero.

4 — Recetario: si ves A → hacé B

Receta 1 — "El que más / menos..." por cada fila → mandadero TOP 1

```
( SELECT TOP 1 f2.fact_cliente
  FROM Item_Factura i2
  JOIN Factura f2 ON (clave de 3)
  WHERE i2.item_producto = p.prod_codigo        -- correlación
  GROUP BY f2.fact_cliente                      -- un grupo por candidato
  ORDER BY SUM(i2.item_cantidad) DESC,          -- "el que más"
        f2.fact_cliente ) AS cliente_top        -- desempate
```

Si no hay ventas devuelve NULL. No uses `ROW_NUMBER()` en un FROM anidado.

Receta 2 — "Los N primeros..., y de ellos solo los que..." → TOP adentro del IN, 2do filtro afuera

```
WHERE p.prod_codigo IN ( SELECT TOP 10 ... ORDER BY ... )   -- 1ro: elijo los 10
  AND (segundo filtro)                                      -- 2do: de esos 10, filtro
```

Receta 3 — "En todos los..." / "en al menos el X%..." → contar y comparar con el total

```
HAVING COUNT(DISTINCT s.stoc_deposito) = ( SELECT COUNT(*) FROM DEPOSITO )

( SELECT COUNT(*) FROM STOCK s WHERE s.stoc_producto = p.prod_codigo AND s.stoc_cantidad > 0 )
   >= 0.75 * ( SELECT COUNT(*) FROM DEPOSITO )
```

El total siempre por subconsulta, nunca un número escrito a mano.

Receta 4 — Comparar años

Años con nombre ("2012 vs 2011") → CASE:

```
SUM(CASE WHEN YEAR(f.fact_fecha) = 2012 THEN i.item_cantidad ELSE 0 END) AS c2012
...
HAVING SUM(CASE ... 2012 ...) > SUM(CASE ... 2011 ...)
```

Años genéricos ("consecutivos", "respecto del anterior", "en algún año") → año ancla:

```
WHERE EXISTS (
   SELECT 1
   FROM Factura f1 JOIN Item_Factura i1 ON (clave de 3)
   WHERE f1.fact_cliente = c.clie_codigo
   GROUP BY YEAR(f1.fact_fecha)                 -- cada grupo = año Y
   HAVING (condición en Y, con agregados de f1/i1)
     AND ( SELECT ... WHERE ... AND YEAR(f2.fact_fecha) = YEAR(f1.fact_fecha) + 1 ) (condición en Y+1)
)
```

3 años → otro mandadero a `+ 2`. Si no hay datos en Y+1 el mandadero da NULL y la condición es falsa.

Receta 5 — "Solo...", "todos sus...", "ninguno..." → NOT EXISTS del contraejemplo

```
NOT EXISTS ( SELECT 1 FROM Item_Factura i
        WHERE (i es de esta factura)
          AND i.item_producto NOT IN ( SELECT comp_producto FROM Composicion ) )
```

⚠️ Un sujeto **sin ninguna fila** cumple el "todos" vacío: exigí que tenga al menos una (`JOIN` o `EXISTS`).

Receta 6 — "Todos, aunque no tengan..." / "0 si no hay"

```
-- el filtro de la tabla dominada va en el ON, NO en el WHERE
FROM Cliente c LEFT JOIN Factura f ON f.fact_cliente = c.clie_codigo AND YEAR(f.fact_fecha) = 2012
-- contar con COUNT(columna), sumar con ISNULL
SELECT COUNT(f.fact_numero), ISNULL(SUM(f.fact_total), 0)
-- en un mandadero, el ISNULL va AFUERA
ISNULL(( SELECT SUM(...) ... ), 0)
```

`x > NULL` nunca es verdadero. Un filtro sobre la tabla **dominante** sí va en el WHERE.

Otra salida (🎙️ Ej. 14): los que tienen `UNION ALL` los que no, con todo en 0.

```
SELECT c.clie_codigo, COUNT(*), AVG(f.fact_total), MAX(f.fact_total)
FROM Cliente c JOIN Factura f ON f.fact_cliente = c.clie_codigo
WHERE YEAR(f.fact_fecha) = 2012
GROUP BY c.clie_codigo
UNION ALL
SELECT c.clie_codigo, 0, 0, 0
FROM Cliente c
WHERE NOT EXISTS (SELECT 1 FROM Factura f2 WHERE f2.fact_cliente = c.clie_codigo AND YEAR(f2.fact_fecha) =
2012)
ORDER BY 2 DESC
```

Receta 7 — "% sobre el total del año" → mandadero correlacionado por año

```
SUM(i.item_cantidad * i.item_precio) * 100.0
  / ( SELECT SUM(i2.item_cantidad * i2.item_precio) FROM ... WHERE YEAR(f2.fact_fecha) = YEAR(f.fact_fecha) )
```

`100.0`, no `100` (división entera).

Receta 8 — "Primero los que..., después el resto"

```
ORDER BY CASE WHEN (condición, típicamente EXISTS(...)) THEN 0 ELSE 1 END, otro_criterio
```

Receta 9 — "Sin subconsultas en el FROM ni CTE"

En vez de FROM anidado para...	Usá
filtrar por un total	`HAVING`
filtrar por un ranking	`IN (SELECT TOP N ...)`
comparar dos períodos	CASE o año ancla
dato de otro nivel	mandadero en el SELECT
promedio de totales por sujeto	`SUM(total) / COUNT(DISTINCT sujeto)`
TOP en una parte de un UNION	`IN (SELECT TOP N ...)` (receta 11)

Receta 11 — "Los N más... y también los N menos..." → UNION ALL, con el TOP adentro del IN

```
SELECT p.prod_codigo, p.prod_detalle, 'Más vendido' AS ranking, (mandadero TOP 1 si piden "el que más")
FROM Producto p
WHERE p.prod_codigo IN (SELECT TOP 10 item_producto FROM Item_Factura
                        GROUP BY item_producto ORDER BY SUM(item_cantidad) DESC, item_producto)
UNION ALL
SELECT p.prod_codigo, p.prod_detalle, 'Menos vendido', (el mismo mandadero)
FROM Producto p
WHERE p.prod_codigo IN (SELECT TOP 10 item_producto FROM Item_Factura
                        GROUP BY item_producto ORDER BY SUM(item_cantidad) ASC, item_producto)
ORDER BY ranking, prod_detalle
```

`SELECT TOP 10 ... ORDER BY ... UNION ...` **no compila**. ⚠️ Empates en el borde (en tu base, 24 productos con 1 unidad): desempatá por código y escribí el supuesto.

5 — Reglas de subconsultas

Dónde	Qué tiene que devolver
`IN (...)` / `NOT IN`	1 columna, 0..N filas
`= (...)`, `>`, `<`	1 valor (si pueden ser varias filas → `IN`)
`SELECT (...) AS col`	1 fila, 1 columna (un agregado sin GROUP BY lo garantiza)
`UPDATE ... SET col = (...)`	1 fila, 1 columna
`FROM (...) alias`	alias obligatorio; la más cara
`EXISTS (...)`	verdadero/falso; `SELECT 1`

- Agregado **sin** GROUP BY sobre nada → **1 fila** (`COUNT` = 0, el resto NULL). **Con** GROUP BY → 0 filas.
- ⚠️ `EXISTS (SELECT COUNT(*) ...)` sin GROUP BY/HAVING es **siempre verdadero**. Usá `WHERE (SELECT COUNT(*) ...) >= N` o `EXISTS (... HAVING COUNT(*) >= N)`.
- `AVG` y `COUNT(col)` ignoran NULL. `= NULL` / `<> NULL` son siempre falsos: `IS [NOT] NULL`.
- Mostrar código + detalle sin repetir → `GROUP BY` (no `DISTINCT`), con el detalle también en el GROUP BY.
- `ORDER BY 2` solo si la columna se muestra; mejor número de columna que alias. En ascendente, los NULL van primero.
- `CASE` sin `ELSE` → NULL; todas las ramas, mismo tipo.
- No metas subconsultas pesadas sobre `Factura` / `Item_Factura` con mucho afuera: se ejecutan una vez por fila.

Operadores de conjunto (`UNION`, `INTERSECT`, `EXCEPT`):

- Mismas columnas en cantidad, orden y tipo; los nombres salen del **primer** SELECT. Columna que falta → `NULL`, `0` o un texto fijo.
- Devuelven el resultado **sin repetidos**, también los repetidos dentro de un mismo conjunto. `UNION ALL` deja todo y es más barato: usalo si sabés que no hay repetidos.
- **Un solo ORDER BY, al final**, ordena todo. TOP en una parte → subconsulta (o `IN`).
- `EXCEPT`: el orden importa. Mezclando operadores, paréntesis (`INTERSECT` va primero).
- Equivalencias: `INTERSECT` ≈ `EXISTS`, `EXCEPT` ≈ `NOT EXISTS`, `UNION` de la misma tabla ≈ `OR`. El operador conviene cuando cada parte tiene **su propio GROUP BY/HAVING**.
- "Todos los meses, aunque no haya ventas" → tabla de meses como dominante: `FROM (SELECT 1 AS mes UNION ALL SELECT 2 ... UNION ALL SELECT 12) m LEFT JOIN Factura f ON MONTH(f.fact_fecha) = m.mes AND YEAR(...) = X`.

ROW_NUMBER: `ROW_NUMBER() OVER (PARTITION BY grupo ORDER BY criterio)` numera sin agrupar y vuelve a 1 en cada grupo. No va en el WHERE: para filtrar por el número, subconsulta en el FROM con alias.

6 — T-SQL

6.1 — Qué objeto, según el verbo

Si dice...	Objeto
"dado X, retornar Y", "usar en un SELECT/WHERE"	**Función** (no modifica datos ni abre transacciones)
"devolver un listado" para usar como tabla	**Función de tabla** (`RETURNS TABLE` , en el FROM con alias)
"directa o indirectamente", "a cualquier nivel"	**Función recursiva**
"corregir", "reasignar", "completar", "recategorizar"	**Procedimiento** (`EXEC`)
"automáticamente", "ante una venta...", "n aplicaciones", "no debe grabarse... error"	**Trigger**
"mantener actualizado / precalculado"	**Tabla + carga inicial + triggers + índice** (6.4)
"consultar rápido", "optimizar filtros por X"	**Índice** clustered sobre lo que se filtra
"simplificar", "ocultar columnas", "acceso restringido"	**Vista**

6.2 — Vistas

- Guarda el SELECT, no los datos. **Misma performance** que la consulta a mano: justificala por **simplicidad o seguridad**, nunca por velocidad.
- Sin `ORDER BY` adentro. Actualizable si es de una sola tabla, sin agrupar. `WITH CHECK OPTION` rechaza filas que la vista no mostraría.
- **Vista indexada** (`WITH SCHEMABINDING` + `CREATE UNIQUE CLUSTERED INDEX`): no admite `MIN` , `MAX` , `AVG` , `TOP` , subconsultas, `DISTINCT` ni `OUTER JOIN` → casi nunca sirve para "precalculado". Decilo en el parcial.

6.3 — Triggers

```
CREATE TRIGGER dbo.tr_nombre ON dbo.Tabla
AFTER INSERT, UPDATE, DELETE        -- o INSTEAD OF ...
AS
BEGIN
  SET NOCOUNT ON;
  ...
END
GO
```

Operación	`inserted`	`deleted`
INSERT	filas nuevas	vacía
DELETE	vacía	filas borradas
UPDATE	versión nueva	versión vieja

1. Se dispara **una vez por sentencia**: `inserted` puede tener muchas filas → `JOIN inserted`, **nunca** `SELECT @x = ... FROM inserted`.
2. `AFTER` = ya pasó (rechazar con ROLLBACK). `INSTEAD OF` = no pasa; vos hacés lo que corresponda.
3. Rechazar: `RAISERROR('mensaje', 16, 1); ROLLBACK TRANSACTION;` (o `THROW 50001, 'mensaje', 1;`).
4. `UPDATE(columna)` → si esa columna estaba en el SET.
5. Sin "antes" en SQL Server: solo `AFTER` e `INSTEAD OF`. Sobre una **vista**, solo `INSTEAD OF`. **Un solo INSTEAD OF** por tabla y evento; AFTER, varios.
6. **Orden:** constraints → modificación → AFTER. Si falla una constraint (FK, PK, CHECK), **el AFTER no corre**. El INSTEAD OF corre antes que las constraints.
7. Corre **dentro de la transacción** de quien lo disparó: `ROLLBACK` deshace **toda** la transacción (también lo hecho antes); `COMMIT` adentro, no.
8. `INSERT ... VALUES (..), (..), (..)` = una instrucción: una transacción, el trigger corre 1 vez y, si falla una fila, no entra ninguna.
9. Antes de programar: **tablas** involucradas (¿"el o los objetos"?), **todos los eventos** (¿también UPDATE? ¿DELETE?) y la **modificación masiva**.
10. Trigger corto y simple (no tiene parámetros y sus errores son poco claros). La lógica larga va en un procedimiento.

(A) Validar:

```
AFTER INSERT, UPDATE
...
IF EXISTS ( (consulta que encuentra una VIOLACIÓN, acotada a lo que tocó inserted) )
BEGIN
   RAISERROR('...', 16, 1);
   ROLLBACK TRANSACTION;
END
```

(B) Mantener precalculado → recalcular lo afectado: 1) claves de `inserted` `UNION` `deleted`; 2) borrarlas del resumen; 3) recalcularlas desde la fuente. (MAX, MIN y "el que más" no se actualizan sumando.)

(C) Propagar: `UPDATE` de la otra tabla `JOIN inserted` (restar) y `JOIN deleted` (devolver).

(D) Borrar en cascada: `INSTEAD OF DELETE` en la cabecera → `DELETE` de los hijos `JOIN deleted`, después `DELETE` de la cabecera `JOIN deleted`. (Con AFTER no se puede: la FK rechaza antes.) Repetir el evento adentro de un INSTEAD OF no lo vuelve a disparar.

(E) Normalizar: `INSTEAD OF INSERT` → `INSERT INTO T SELECT UPPER(...), 1, GETDATE() ... FROM inserted`. Distinto de un DEFAULT: "no te lo pido, siempre pongo este".

(F) Vista de dos tablas: `INSTEAD OF INSERT` en la vista → `INSERT` en la tabla padre `WHERE NOT EXISTS` (con `DISTINCT`), y después en la hija.

Único salvo NULL: `UNIQUE` admite un solo NULL → trigger con `WHERE col IN (SELECT col FROM inserted) GROUP BY col HAVING COUNT(*) > 1`.

6.4 — Checklist "mantener precalculado"

1. **Tabla** con la granularidad pedida → PK `(anio, mes, producto)`.

2. **Índice**: PK **clustered** empezando por las columnas del filtro.
3. **Carga inicial** de lo que ya existe (procedimiento o script).
4. **Triggers en todas las tablas fuente**: `Item_Factura` (INSERT/UPDATE/DELETE) y `Factura` (UPDATE de fecha o cliente).
5. **Supuestos** escritos: desempates, grupo que se queda sin ventas.

6.5 — Función recursiva

```
CREATE FUNCTION dbo.fn_precio_real (@prod CHAR(8)) RETURNS DECIMAL(12,2)
AS
BEGIN
  IF NOT EXISTS ( SELECT 1 FROM Composicion WHERE comp_producto = @prod )   -- base
    RETURN ( SELECT prod_precio FROM Producto WHERE prod_codigo = @prod );
  RETURN ( SELECT SUM(c.comp_cantidad * dbo.fn_precio_real(c.comp_componente))  -- recursivo
        FROM Composicion c WHERE c.comp_producto = @prod );
END
```

Empleados a cargo directa o indirectamente: `COUNT(directos) + SUM(dbo.fn(cada directo))`. Con cursor:

```
DECLARE @emp NUMERIC(6), @total INT = 0;
DECLARE c_sub CURSOR FOR
  SELECT empl_codigo FROM Empleado WHERE empl_jefe = @jefe;
OPEN c_sub;
FETCH NEXT FROM c_sub INTO @emp;
WHILE @@FETCH_STATUS = 0
BEGIN
  SET @total = @total + 1 + dbo.fn_cant_empleados(@emp);
  FETCH NEXT FROM c_sub INTO @emp;
END
CLOSE c_sub;
DEALLOCATE c_sub;
RETURN @total;
```

6.6 — Reglas de funciones

- Termina en `RETURN` en **todos** los caminos. Variables inicializadas.
- `SELECT @v = col`: sin filas conserva el valor anterior; con varias se queda con una, sin error.
- Decimales con punto. `BREAK` sale del WHILE.
- Se llama siempre con esquema: `dbo.fn(...)`. Se puede usar en SELECT, WHERE, HAVING, GROUP BY, ORDER BY, SET y VALUES.
- Se ejecuta una vez por fila: pasale los datos que ya leíste, no la PK para que vuelva a buscar.

6.7 — Stored procedure

```
CREATE PROCEDURE dbo.sp_nombre (@param1 CHAR(8), @monto DECIMAL(12,2), @resultado INT OUTPUT)
AS
BEGIN
  SET NOCOUNT ON;
  IF @param1 IS NULL OR @monto <= 0     -- 1) defensivo, ANTES de la transacción
  BEGIN
    SET @resultado = 1;
    RETURN;
  END
  BEGIN TRY
    BEGIN TRANSACTION;
    UPDATE ... WHERE clave = @param1;
    IF @@ROWCOUNT = 0
    BEGIN
      ROLLBACK TRANSACTION;       -- no corta el SP
      SET @resultado = 2;
      RETURN;                     -- por eso, RETURN
    END
    ...
    COMMIT TRANSACTION;
    SET @resultado = 0;
  END TRY
  BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    SET @resultado = 99;
    PRINT ERROR_MESSAGE();
  END CATCH
END
GO
DECLARE @r INT;
EXEC dbo.sp_nombre '00001705', 10, @r OUTPUT;
SELECT @r;
```

- `ROLLBACK` no corta el SP ni deshace variables → después de cada uno, `RETURN`.
- Un UPDATE/DELETE que no afecta filas no da error → `@@ROWCOUNT`.
- `@@IDENTITY` = último autonumérico. Alta o modificación → `MERGE`.

7 — Checklist antes de entregar

Consultas:

- [ ] ¿El FROM arranca en el **sujeto**?
- [ ] ¿Uní `Item_Factura` con `Factura` por las **tres** columnas?
- [ ] ¿Algún `SUM` / `AVG` después de unir **dos tablas hijas**? → mandadero.
- [ ] ¿Los mandaderos que pueden no encontrar nada tienen `ISNULL(..., 0)` **afuera**?
- [ ] ¿Hay `TOP` + segundo filtro? → ¿está el TOP **adentro** del `IN`?
- [ ] ¿Las comparaciones de porcentaje usan `1.2 *` o `100.0` (no división entera)?
- [ ] ¿Escribí en un comentario los **supuestos** (desempates, interpretación de "primeros", año)?
- [ ] ¿Cumple las restricciones del enunciado (sin subconsulta en FROM, sin CTE)?
- [ ] ¿Algún LEFT JOIN tiene un filtro sobre la tabla dominada en el `WHERE`? → pasalo al `ON`.
- [ ] ¿Algún `EXISTS (SELECT COUNT/SUM ...)` sin GROUP BY? → es siempre verdadero.
- [ ] ¿Algún "todos" que se cumple vacío (sujeto sin filas)?

Objetos:

- [ ] ¿Elegí el objeto por el verbo (6.1)? ¿Justifiqué por qué no una vista?
- [ ] ¿El trigger funciona con **varias filas** a la vez (sin `SELECT @x = ... FROM inserted`)?
- [ ] ¿Miré `inserted` **y** `deleted`?
- [ ] ¿Cubrí **todas** las tablas fuente que pueden cambiar el resultado?
- [ ] ¿Hay carga inicial de los datos existentes?
- [ ] ¿Hay `GO` antes de cada `CREATE`?
- [ ] ¿La función termina en `RETURN` en todos los caminos? ¿Las variables están inicializadas?
- [ ] ¿Abrí un cursor? → `CLOSE` + `DEALLOCATE`.
- [ ] En el SP: ¿después de cada `ROLLBACK` hay un `RETURN`? ¿El parámetro `OUTPUT` queda cargado en todos los caminos?
- [ ] ¿Justifiqué la vista por simplicidad o seguridad, **no** por performance?
- [ ] ¿El índice empieza por las columnas del filtro?